

Equivariant Systems Theory and Observer Design for Autonomous Systems

R. Mahony J. Trumpf T. Hamel



Australian
National
University



Module M17, EECI International Graduate School on Control
Zaragoza, 22 June – 26 June 2026

Outline

- 1 Sensor errors
- 2 Dealing with velocity bias
- 3 Dealing with measurement delay
- 4 Numerical integration for observers

Outline

- 1 Sensor errors
- 2 Dealing with velocity bias
- 3 Dealing with measurement delay
- 4 Numerical integration for observers

Basic sensor model

Conceptually, we have modelled sensors and their measurements as (possibly time-varying) maps

$$h: [t_0, t_1] \times \mathcal{X} \rightarrow \mathcal{Y}$$

from the state space $\mathcal{X}$ to an output space $\mathcal{Y}$, e.g. $\mathcal{X} = \text{SO}(3)$ and $\mathcal{Y} = \text{S}^2 \subset \mathbb{R}^3$ for directional measurements $y(t) = R(t)^\top \dot{y}$ in the attitude observer problem where $h(t, R) = R^\top \dot{y}$.

This means that we assume that measurements of the output signal

$$y(t) = h(t, x(t))$$

are available instantaneously at any time and without any measurement error.

Models for real sensors: bias and noise

In reality, measurements from sensors are never exact. Our model is then a *disturbed* output map

$$\tilde{h}: [t_0, t_1] \times \mathcal{X} \rightarrow \mathcal{Y}.$$

A very common situation (when $\mathcal{Y}$ is a vector space) is that of *additive noise*

$$\tilde{h}(t, x) = h(t, x) + b(t) + n(t),$$

where we think of $n(t)$ as a *zero mean noise process* and of $b(t)$ as a time-varying *bias*.

Models for real sensors: unavailability and quantization

Unavailability of measurements at particular time instances is typically modelled by augmenting the output space $\tilde{\mathcal{Y}} = \mathcal{Y} \cup \{\emptyset\}$ with a special symbol $\emptyset$ that indicates absence or invalidity of a measurement, i.e. $y(t) = h(t, x(t)) = \emptyset$.

Quantization of measurements occurs where the output space $\mathcal{Y}$ is replaced by a *finite* space $\tilde{\mathcal{Y}}$. For example, with $\mathcal{Y} = \mathbb{R}$ and $\tilde{\mathcal{Y}} = \mathbb{Z}$, $\tilde{h}(t, x) = \text{floor}(h(t, x))$. General *quantization maps* can be quite complex.

In both cases the disturbed output map has the form

$$\tilde{h}: [t_0, t_1] \times \mathcal{X} \rightarrow \tilde{\mathcal{Y}}.$$

Models for real sensors: delay

Another very common type of measurement error is *delay*

$$\tilde{h}(t, x) = h(t - \tau, x),$$

where τ is the (constant) measurement delay.

Delay corresponds to a modification of the time set $[t_0, t_1]$. For the situation of constant delay τ we have the modified output map

$$\tilde{h}: [t_0 + \tau, t_1 + \tau] \times \mathcal{X} \rightarrow \mathcal{Y}$$

Models for real sensors: sampling

Yet another source of error is *sampling* where we only have a sequence of measurements of the output signal $y(t)$,

$$y_{t_1}, y_{t_2}, \dots, y_{t_N}$$

at the sample times t_i . Here, $y_{t_i} = y(t_i) = h(t_i, x(t_i))$.

Errors, or at least additional design complexities, can for example result from *asynchronous sampling*, *out-of-order* measurements, *multi-rate* sampling or *time-stamping* errors.

Conceptually, sampling can be modelled as *time discretization*, i.e. our sampled output map has the structure

$$\tilde{h}: (t_1, t_2, \dots, t_N) \times \mathcal{X} \rightarrow \mathcal{Y}.$$

Outline

- 1 Sensor errors
- 2 Dealing with velocity bias
- 3 Dealing with measurement delay
- 4 Numerical integration for observers

Velocity bias

A common fault with velocity measurement systems is the corruption of the true signal by a slowly time-varying bias.

Thus, the measurement u_y of $u \in \mathbb{V}$ is corrupted by a constant offset

$$u_y = u + b$$

$b \in \mathbb{V}$ is a constant unknown bias.

In practice, bias is caused by temperature variation, vibration, drifting voltage bus levels, etc. In order to treat it as constant these variations need to be significantly slower than the system dynamics so that there is a frequency separation between the signals u and b .

Problem formulation

The system model considered is

$$\begin{aligned}\dot{R} &= R\Omega_{\times} \\ \dot{B} &= 0 \\ U_y &= \Omega_{\times} + B \\ y_i &= R^{\top} \dot{y}_i\end{aligned}$$

on $\text{SO}(3)$. The observer that we consider is

$$\begin{aligned}\dot{\hat{R}} &= \hat{R} \left(U_y - \hat{B} \right) - k_P \text{grad}_{\hat{R}} \ell_{\dot{y}}(\hat{R}, y) \\ \dot{\hat{B}} &= \beta_B(\hat{R}, \dot{y}_1, \dots, \dot{y}_p, y_1, \dots, y_p)\end{aligned}$$

where $\hat{B} \in \mathfrak{so}(3)$ and $\ell_{\dot{y}}(\hat{R}, y) = \sum_{i=1}^p k_i \ell_{\dot{y}_i}(\hat{R}, y_i) = \sum_{i=1}^p \frac{k_i}{2} \|\hat{R} y_i - \dot{y}_i\|^2$.

Errors

The group error that we use is

$$E_l = \hat{R}R^\top.$$

A new bias error is introduced

$$\tilde{B} = \hat{B} - B.$$

Similar to the error convergence analysis of the $SO(3)$ gradient observer design the easiest analysis of bias compensation uses a Lyapunov function,

$$\mathcal{L} = \ell_{\dot{y}}(E_l, \dot{y}) + \frac{1}{2\gamma} \langle \tilde{B}, \tilde{B} \rangle_{\mathfrak{so}(3)}.$$

Lyapunov analysis

Computing the time derivative of the Lyapunov function along trajectories gives

$$\begin{aligned} \frac{d}{dt} \mathcal{L} = & D_{E_l} \ell_{\dot{y}}(E_l, \dot{y}) \cdot [-k_P \text{grad}_{E_l} \ell_{\dot{y}}(E_l, \dot{y})] \\ & + \langle \text{grad}_{\hat{R}} \ell_{\dot{y}}(\hat{R}, y) \hat{R}^\top, -\text{Ad}_{\hat{R}} \tilde{B} \rangle + \frac{1}{\gamma} \langle \tilde{B}, \beta_B \rangle \end{aligned}$$

and thus the choice

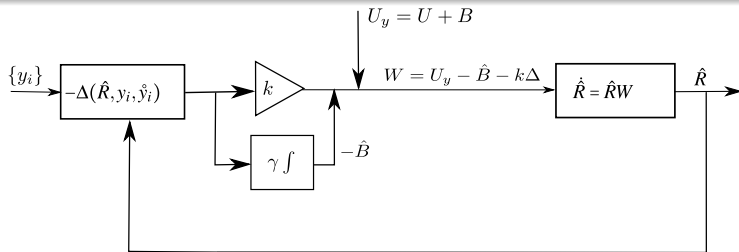
$$\beta_B := \gamma \hat{R}^\top \text{grad}_{\hat{R}} \ell_{\dot{y}}(\hat{R}, y)$$

leads to

$$\frac{d}{dt} \mathcal{L} = -k \| \text{grad}_{E_l} \ell_{\dot{y}}(E_l, \dot{y}) \|^2$$

and a LaSalle type analysis for the observer error convergence.

Block Diagram



The observer model that we consider is

$$\dot{\hat{R}} = \hat{R} \left(U_y - \hat{B} - k_P \alpha(\hat{R}, y, \dot{y}) \right)$$

$$\dot{\hat{B}} = \gamma \alpha(\hat{R}, y, \dot{y})$$

where $\alpha(\hat{R}, y, \dot{y}) = \sum_{i=1}^p k_i (\hat{y}_i \times y_i)_\times$.

Outline

- 1 Sensor errors
- 2 Dealing with velocity bias
- 3 Dealing with measurement delay**
- 4 Numerical integration for observers

Motivation: measurement delay

- Measurement delays occur in practice due to
 - ★ physical properties of sensors or the environment
 - ★ internal signal processing of sensors
 - ★ (pre-)filtering of sensor measurements
 - ★ communication delays
- Practical examples with significant sensor delays
 - ★ Image processing (X0 msec)
 - ★ GPS (X00 msec)
 - ★ Star trackers or Earth sensors (0.1 - 2 sec)
 - ★ Communication delay VICON or OptiTrack (up to X00 msec)
- Measurement delays can cause instability in closed loop if not compensated properly.

General approaches for compensating sensor delay

- Lyapunov-Krasovskii methods

Pros: simple design

Cons: complicated analysis, requires careful and conservative gain tuning

- Stochastic filtering with out-of-sequence measurements

Pros: flexible in dealing with sampled and delayed data

Cons: significant memory and processing requirements, no stability proof

- Observer-predictor cascades

Pros: guaranteed stability, low computational load

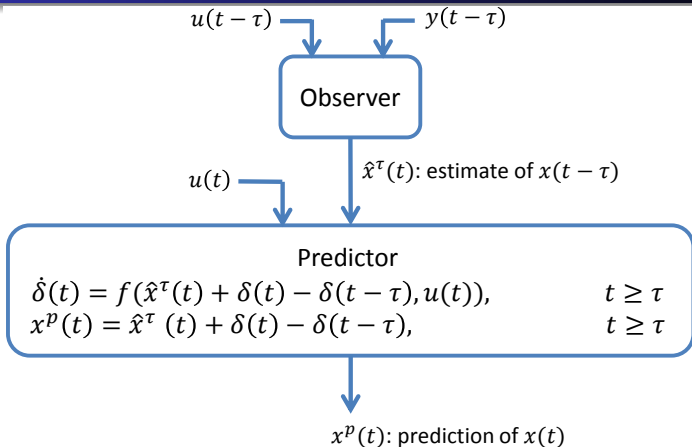
Cons: relatively large memory requirement for sensor buffering, newer than other methods and hence not as well developed, not always easy with off-the-shelf observers

- Predictor-observer cascades

Pros: guaranteed stability, low computational load, easy to implement with off-the-shelf observers

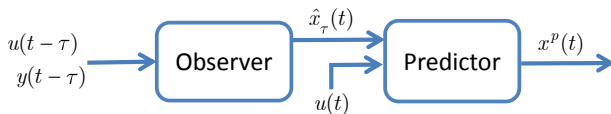
Cons: only suits special classes of output models, sensitive to biased/noisy/delayed input measurements

Observer-predictor cascades



The proposed predictor is *recursive* and it is *generic* in the sense that it can be combined with any asymptotic observer (or filter).

Generalization to invariant systems on Lie groups



System on $\mathbb{R}^n$: $\dot{x}(t) = f(x(t), u(t))$

System on G : $\dot{X}(t) = X(t)u(t)$

Predictor

$$\begin{cases} \dot{\delta}(t) = f(\hat{x}^\tau(t) + \delta(t) - \delta(t - \tau), u(t)) \\ x^p(t) = \hat{x}^\tau(t) + \delta(t) - \delta(t - \tau) \end{cases}$$

Predictor

$$\begin{cases} \dot{\Delta}(t) = \Delta(t)u(t) \\ X^p(t) = \hat{X}^\tau(t)\Delta(t - \tau)^{-1}\Delta(t) \end{cases}$$

In the Lie group scenario, the convergence rate is ∞ for arbitrary large τ .

A. Khosravian, J. Trumpf, R. Mahony, and T. Hamel. State estimation for invariant systems on Lie groups with delayed output measurements. *Automatica*, 68:254–265, 2016

Simulation assumptions

We use the geometric observer developed in [Hua et al., CDC 2011] to obtain the estimates $\hat{R}_\tau(t)$ and $\hat{v}_\tau(t)$.

The vehicle performs circular manoeuvres with the radius of 5 meters and the angular velocity of 45 (deg/s) in the horizontal plane.

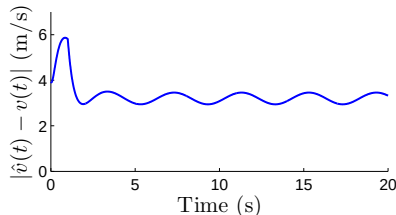
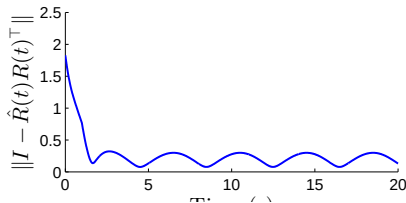
Velocity measurement delay of $\tau = 1$ (s).

Initial attitude of the vehicle: roll 80 (deg), pitch 10 (deg), yaw 0 (deg).

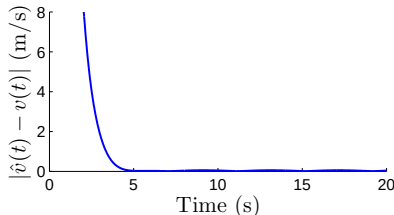
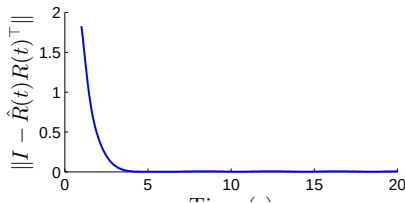
Initial condition of observer: $(\hat{R}_\tau(0), \hat{v}_\tau(0)) = (I, 0)$.

Initial condition of predictor: $(\Delta_0, \delta_0) = (I, 0)$.

Simulation I: observer only vs. observer-predictor

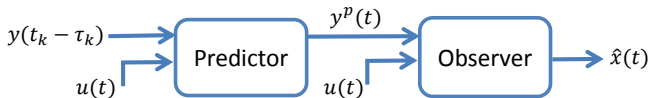


Estimation error of the SE(3) observer of [Hua et al., CDC 2011] when fed with $v(t - \tau)$.



Estimation error of the observer-predictor approach fed with $v(t - \tau)$.

Predictor-observer cascades



System on G :	$\dot{X}(t) = X(t)u(t)$	Output predictor
Ideal output:	$y(t) = h(X(t), \dot{y})$	$\begin{cases} \dot{\Delta}(t) = \Delta(t)u(t) \\ y^p(t) = h(\Delta(t_k - \tau_k)^{-1}\Delta(t), y(t_k - \tau_k)) \end{cases}$
Sampled and delayed output:	$y(t_k - \tau_k)$	

A. Khosravian, J. Trumpf, R. Mahony, and T. Hamel. Recursive attitude estimation in the presence of multi-rate and multi-delay vector measurements. In Proceedings of the American Control Conference (ACC), pages 3199–3205, 2015

Experimental verification

Autopilot System: ArduPilot/APM

Vehicle: Plane

The estimation algorithm (EKF) runs at 50 Hz.

GPS measurements (velocity and position) are 220 (ms) delayed.

Magnetometer measurements are 40 (ms) delayed.

Predictor also compensates the delay due to processing time of EKF.



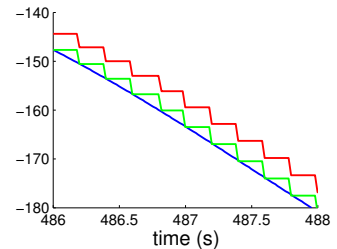
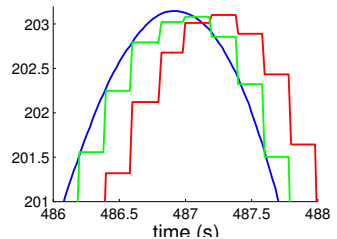
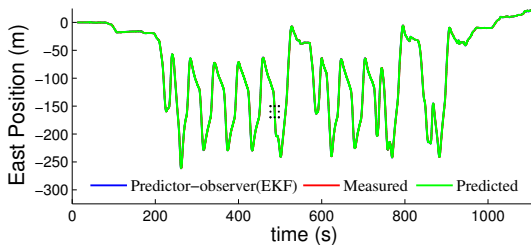
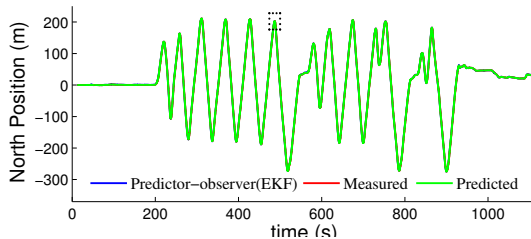
J. Pittar, S. O'Brien, A. Khosravian



A. Kroll, A. Tridgell, S. O'Brien

Experimental results: predictor-observer on SO(3)

Controller is fed with the predictor-observer for the whole flight.



Outline

- 1 Sensor errors
- 2 Dealing with velocity bias
- 3 Dealing with measurement delay
- 4 Numerical integration for observers

Euler vs Lie group Euler

The observer model that we consider is

$$\begin{aligned}\dot{\hat{R}} &= \hat{R} \left(U_y - \hat{B} - k_P \alpha(\hat{R}, y, \dot{y}) \right) \\ \dot{\hat{B}} &= \gamma \alpha(\hat{R}, y, \dot{y})\end{aligned}$$

where $\alpha(\hat{R}, y, \dot{y}) = \sum_{i=1}^p k_i (\hat{y}_i \times y_i)_\times$.

How do we implement this in MATLAB/Octave/Python?